Enterprise AI Solution Report

Requirement Analysis

Based on the requirement, here's a breakdown of what can be inferred about the system and its components:

Project Objective

The project objective is to build an Employee Management System. This suggests that the system will aim to manage employee-related data and processes effectively.

Key Features

The following features are identified as essential for the system:

1. **Login**: This feature allows authorized users to access the system.
2. **Attendance**: This feature will likely be used to track employees' presence or absence.
3. **Leave Management**: This feature will enable administrators to manage employee leave requests, approvals, and tracking.
4. **Reports**: This feature provides insights into various aspects of the employee management, such as attendance patterns, leave balances, etc.

Stakeholders

The stakeholders for this system are likely:

1. **Employees**: They will use the attendance and leave management features to track their work hours and request leaves.
2. **Administrators/Supervisors**: They will use the login feature to access the system, manage employee data, and approve leave requests.
3. **HR Department**: They might be involved in the leave management process, reporting, and ensuring data accuracy.

Functional Requirements

Based on the features identified, some functional requirements that can be inferred are:

1. User authentication and authorization (Login)
2. Data tracking for attendance and leave (Attendance and Leave Management)
3. Reporting capabilities (Reports)
4. Database integration to store employee data
5. API integration for REST-based communication

Assumptions

Some assumptions can be made about the system based on this requirement:

1. The system will require a database schema to store employee data, attendance records, leave balances, and other relevant information.
2. The API will need to handle user authentication, attendance tracking, leave requests, and reporting.
3. The system might include additional features not explicitly mentioned in the requirement.

Next Steps

To move forward with this project, it's essential to:

1. Define a more detailed database schema.
2. Design the REST API architecture, including endpoints for login, attendance, leave management, and reports.
3. Plan the implementation of key features, such as user authentication, data tracking, and reporting.
4. Identify potential roadblocks or technical challenges that might arise during development.

By following these steps, you can begin to flesh out the requirements and start building the Employee Management System.

User Stories

Here are some sample Agile user stories for the Employee Management System:

Login Feature

* User Story 1:
+ As a valid user, I want to be able to log in to the system using my username and password.
+ Acceptance Criteria:
- The login page displays correctly.
- The system validates the username and password.
- A successful login returns an authorized token.
* User Story 2:
+ As a user, I want to be able to log out of the system securely using my session token.
+ Acceptance Criteria:
- The logout process ends the current session.
- The system redirects to the login page if the session is invalid.

Attendance Feature

* User Story 3:
+ As an employee, I want to be able to mark my attendance for a given day.
+ Acceptance Criteria:
- The attendance page displays correctly.
- The system validates the employee's ID and date.
- The employee can select their attendance status (present/absent).
* User Story 4:
+ As an administrator, I want to be able to view all employees' attendance for a given period.
+ Acceptance Criteria:
- The attendance report page displays correctly.
- The system retrieves the attendance data from the database.

Leave Management Feature

* User Story 5:
+ As an employee, I want to be able to request a leave for a specific date range.
+ Acceptance Criteria:
- The leave request form displays correctly.
- The system validates the employee's ID and dates.
- The employee can select their leave type (annual/medical/etc.).
* User Story 6:
+ As an administrator, I want to be able to approve/reject leave requests from employees.
+ Acceptance Criteria:
- The leave management page displays correctly.
- The system sends notifications to the employee regarding the status of their request.

Reports Feature

* User Story 7:
+ As an administrator, I want to be able to generate a report on all employees' attendance for a given period.
+ Acceptance Criteria:
- The report page displays correctly.
- The system retrieves the attendance data from the database and generates the report.
* User Story 8:
+ As an administrator, I want to be able to view a list of all leave requests submitted by employees

for a given period.
+ Acceptance Criteria:
- The leave request report page displays correctly.
- The system retrieves the leave data from the database and generates the report.

Database Requirements

* User Story 9:
+ As an administrator, I want to be able to configure the database schema for the Employee Management System.
+ Acceptance Criteria:
- The database schema is properly configured.
- The system can store employee data correctly.
* User Story 10:
+ As an administrator, I want to be able to perform CRUD (Create, Read, Update, Delete) operations on employee data in the database.
+ Acceptance Criteria:
- The CRUD operations are performed correctly.

REST API Requirements

* User Story 11:
+ As a developer, I want to be able to create REST APIs for the Employee Management System.
+ Acceptance Criteria:
- The REST APIs are properly configured and tested.
- The system can retrieve and update employee data using the REST APIs.
* User Story 12:
+ As an administrator, I want to be able to use the REST APIs to integrate with other systems or services.
+ Acceptance Criteria:
- The REST APIs return valid responses for all requests.
- The system can handle API requests correctly.

Note: These user stories are just examples and may need to be adjusted based on the specific requirements of your project.

API Design

Employee Management System REST API Design

Overview

This document outlines the design of a REST API for an employee management system. The system will provide endpoints for user authentication, attendance tracking, leave management, and reports.

Database Schema

The database schema consists of the following tables:

* **Employees**
* `id` (primary key)
* `name`
* `email`
* `password` (hashed)
* **Attendances**
* `id` (primary key)
* `employee_id` (foreign key referencing Employees table)
* `date`
* `present`
* **Leaves**
* `id` (primary key)
* `employee_id` (foreign key referencing Employees table)
* `type` (leave type, e.g., annual leave, sick leave)
* `start_date`
* `end_date`

REST API Endpoints

Authentication

* **Endpoint:** `/api/auth/login`
* **Method:** POST
* **Request:**
* `body`:
* `username`: string (employee email)
* `password`: string (employee password)
* **Response:**
* `status_code`: 200
* `token`: string (authentication token)

Attendance

* **Endpoint:** `/api/attendances`
* **Method:** POST
* **Request:**
* `body`:
* `employee_id`: integer (employee ID)
* `date`: date
* `present`: boolean (true for present, false for absent)
* **Response:**
* `status_code`: 200

* `id`: integer (attendance ID)

Leave Management

* **Endpoint:** `/api/leaves`
* **Method:** POST
* **Request:**
* `body`:
* `employee_id`: integer (employee ID)
* `type`: string (leave type, e.g., annual leave, sick leave)
* `start_date`: date
* `end_date`: date
* **Response:**
* `status_code`: 200
* `id`: integer (leave ID)

Reports

* **Endpoint:** `/api/reports`
* **Method:** GET
* **Request: None**
* **Response:**
* `status_code`: 200
* `data`: array of report data (e.g., attendance, leave details)

Example Use Cases

Login Example

```bash
curl -X POST \
http://localhost:8080/api/auth/login \
-H 'Content-Type: application/json' \
-d '{"username": "john.doe@example.com", "password": "password123"}'
```

Response:
```json
{
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Ikpv
aGFuIjoiMjMwfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c"
}
```

Attendance Example

```bash
curl -X POST \
http://localhost:8080/api/attendances \
-H 'Content-Type: application/json' \
-d '{"employee_id": 1, "date": "2023-02-20", "present": true}'
```

Response:
```json
{
```

"id": 1
}
```

#### Leave Example

```bash
curl -X POST \
http://localhost:8080/api/leaves \
-H 'Content-Type: application/json' \
-d '{"employee_id": 1, "type": "annual leave", "start_date": "2023-02-20", "end_date": "2023-03-01"}'
```

Response:
```json
{
"id": 1
}
```

Report Example

```bash
curl -X GET \
http://localhost:8080/api/reports
```

Response:
```json
[
{
"employee_id": 1,
"date": "2023-02-20",
"present": true
},
{
"employee_id": 2,
"date": "2023-02-21",
"present": false
}
]
```

API Documentation

The API documentation will be generated using Swagger/OpenAPI. The documentation will include the following:

* Endpoint descriptions
* Request and response body schema definitions
* Authentication mechanisms (e.g., token-based authentication)
* Error handling guidelines

Note: This is a basic design and may require modifications based on specific requirements and constraints.
```

Security Considerations

The API should implement proper security measures, such as:

* Token-based authentication for user authentication
* Input validation and sanitization to prevent SQL injection and cross-site scripting (XSS) attacks
* Encryption for sensitive data transmission (e.g., employee passwords)
* Secure password storage using a secure hashing algorithm

Database Security Considerations

The database should implement proper security measures, such as:

* Using a secure connection protocol (e.g., SSL/TLS)
* Encrypting sensitive data in transit and at rest
* Implementing access controls and permissions to restrict database access
* Regularly updating and patching the database software